Exercicio 6 - Bases Reais com ELMs

Eduardo Rios de Andrade Sousa - 2022087105

Codigo Fonte

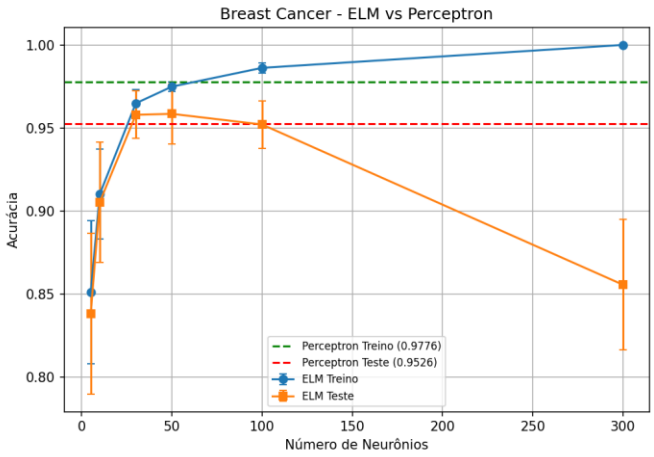
```
import numpy as np import matplotlib.pyplot as
plt from sklearn.preprocessing import
MinMaxScaler from sklearn.datasets import
load_breast_cancer import urllib.request
import os

def tanh(x):
return np.tanh(x)

def elm_train(X, y, n_hidden):
n_features = X.shape[1] W =
np.random.randn(n_features, n_hidden) b
= np.random.randn(1, n_hidden) H =
tanh(X @ W + b) H_pinv =
np.linalg.pinv(H) beta = H_pinv @ y
return W, b, beta

def elm_predict(X, W, b,
beta): H = tanh(X @ W + b)
y_pred = H @ beta return
np.sign(y_pred)

def accuracy(y_true, y_pred): return
np.mean(y_true.flatten() == y_pred.flatten())

def perceptron_train(X, y, lr=0.01, max_iter=1000):
n = X.shape[1] w =
np.zeros(n) b_p = 0.0
for _ in
range(max_iter): for i
in range(len(X)):
pred = np.sign(X[i] @ w + b_p)
if pred == 0:
pred = -1 if pred !=
y[i, 0]: w += lr * y[i,
0] * X[i] b_p += lr *
y[i, 0] return w, b_p

def perceptron_predict(X, w, b_p):
pred = np.sign(X @ w +
b_p) pred[pred == 0] = -1
return pred.reshape(-1, 1)

# --- Breast Cancer --data =
load_breast_cancer() X_bc = data.data y_bc
= (data.target * 2 - 1).reshape(-1, 1)
scaler_bc = MinMaxScaler() X_bc =
scaler_bc.fit_transform(X_bc)

neurons_real = [5, 10, 30, 50, 100, 300]
n_runs = 10

bc_train_accs = {n: [] for n in neurons_real}
bc_test_accs = {n: [] for n in neurons_real}

for run in range(n_runs):
perm = np.random.permutation(len(X_bc)) split = int(0.7 * len(X_bc))
X_tr, X_te = X_bc[perm[:split]], X_bc[perm[split:]] y_tr, y_te =
y_bc[perm[:split]], y_bc[perm[split:]] for n_h in neurons_real: W, b,
beta = elm_train(X_tr, y_tr, n_h)
bc_train_accs[n_h].append(accuracy(y_tr, elm_predict(X_tr, W, b,
beta))) bc_test_accs[n_h].append(accuracy(y_te, elm_predict(X_te, W, b,
beta)))

p_train_accs_bc, p_test_accs_bc = [], []
for run in range(n_runs):
perm = np.random.permutation(len(X_bc)) split = int(0.7 * len(X_bc))
X_tr, X_te = X_bc[perm[:split]], X_bc[perm[split:]] y_tr, y_te =
y_bc[perm[:split]], y_bc[perm[split:]] w, b_p = perceptron_train(X_tr,
y_tr) p_train_accs_bc.append(accuracy(y_tr, perceptron_predict(X_tr, w,
b_p)))
```

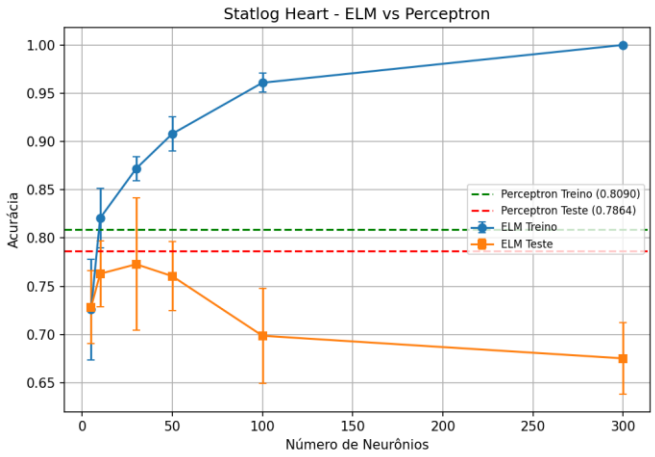
```
p_test_accs_bc.append(accuracy(y_te, perceptron_predict(X_te, w, b_p)))

# --- Statlog Heart --heart_path = 'heart.dat' url =
"https://archive.ics.uci.edu/ml/machine-learning-databases/statlog/heart/heart.dat" if
not os.path.exists(heart_path): urllib.request.urlretrieve(url, heart_path)

heart_data = np.loadtxt(heart_path) X_heart = heart_data[:, :-
1] y_heart = np.where(heart_data[:, -1] == 1, -1, 1).reshape(-
1, 1) scaler_h = MinMaxScaler() X_heart =
scaler_h.fit_transform(X_heart)

h_train_accs = {n: [] for n in neurons_real}
h_test_accs = {n: [] for n in neurons_real}

for run in range(n_runs):
perm = np.random.permutation(len(X_heart)) split = int(0.7 *
len(X_heart)) X_tr, X_te = X_heart[perm[:split]],
X_heart[perm[split:]] y_tr, y_te = y_heart[perm[:split]],
y_heart[perm[split:]] for n_h in neurons_real: W, b, beta =
elm_train(X_tr, y_tr, n_h) h_train_accs[n_h].append(accuracy(y_tr,
elm_predict(X_tr, W, b, beta))) h_test_accs[n_h].append(accuracy(y_te,
elm_predict(X_te, W, b, beta)))

p_train_accs_h, p_test_accs_h = [], []
for run in range(n_runs):
perm = np.random.permutation(len(X_heart)) split = int(0.7 *
len(X_heart)) X_tr, X_te = X_heart[perm[:split]],
X_heart[perm[split:]] y_tr, y_te = y_heart[perm[:split]],
y_heart[perm[split:]] w, b_p = perceptron_train(X_tr, y_tr)
p_train_accs_h.append(accuracy(y_tr, perceptron_predict(X_tr, w,
b_p))) p_test_accs_h.append(accuracy(y_te, perceptron_predict(X_te, w,
b_p)))

# --- Graficos --fig, axes = plt.subplots(1,
2, figsize=(14, 5))

tr_m = [np.mean(bc_train_accs[n]) for n in neurons_real]
tr_s = [np.std(bc_train_accs[n]) for n in neurons_real]
te_m = [np.mean(bc_test_accs[n]) for n in neurons_real]
te_s = [np.std(bc_test_accs[n]) for n in neurons_real]

axes[0].errorbar(neurons_real, tr_m, yerr=tr_s, fmt='o-', label='ELM Treino',
capsize=3) axes[0].errorbar(neurons_real, te_m, yerr=te_s, fmt='s-', label='ELM Teste',
capsize=3) axes[0].axhline(y=np.mean(p_train_accs_bc), color='green', linestyle='--',
label=f'Perceptron Treino ({np.mean(p_train_accs_bc):.4f})')
axes[0].axhline(y=np.mean(p_test_accs_bc), color='red', linestyle='--',
label=f'Perceptron Teste ({np.mean(p_test_accs_bc):.4f})') axes[0].set_xlabel('Numero
de Neuronios') axes[0].set_ylabel('Acuracia') axes[0].set_title('Breast Cancer - ELM vs
Perceptron') axes[0].legend(fontsize=8) axes[0].grid(True)

tr_mh = [np.mean(h_train_accs[n]) for n in neurons_real]
tr_sh = [np.std(h_train_accs[n]) for n in neurons_real]
te_mh = [np.mean(h_test_accs[n]) for n in neurons_real]
te_sh = [np.std(h_test_accs[n]) for n in neurons_real]

axes[1].errorbar(neurons_real, tr_mh, yerr=tr_sh, fmt='o-', label='ELM Treino',
capsize=3) axes[1].errorbar(neurons_real, te_mh, yerr=te_sh, fmt='s-', label='ELM Teste',
capsize=3) axes[1].axhline(y=np.mean(p_train_accs_h), color='green', linestyle='--',
label=f'Perceptron Treino ({np.mean(p_train_accs_h):.4f})')
axes[1].axhline(y=np.mean(p_test_accs_h), color='red', linestyle='--', label=f'Perceptron
Teste ({np.mean(p_test_accs_h):.4f})') axes[1].set_xlabel('Numero de Neuronios')
axes[1].set_ylabel('Acuracia') axes[1].set_title('Statlog Heart - ELM vs Perceptron')
axes[1].legend(fontsize=8) axes[1].grid(True)

plt.tight_layout()
plt.savefig('exercicio2_real.png', dpi=150)
plt.show()
```

Resultados - Saida do Terminal

```
=== EXERCICIO 2 - Breast Cancer ===

  Neuronios=   5 | Treino=0.8510+-0.0433 | Teste=0.8380+-0.0484

  Neuronios=  10 | Treino=0.9101+-0.0271 | Teste=0.9053+-0.0361

  Neuronios=  30 | Treino=0.9648+-0.0083 | Teste=0.9579+-0.0143

  Neuronios=  50 | Treino=0.9749+-0.0028 | Teste=0.9585+-0.0182

  Neuronios= 100 | Treino=0.9862+-0.0030 | Teste=0.9520+-0.0143

  Neuronios= 300 | Treino=1.0000+-0.0000 | Teste=0.8556+-0.0392


--- Perceptron - Breast Cancer ---

  Perceptron | Treino=0.9776+-0.0161 | Teste=0.9526+-0.0232

=== EXERCICIO 2 - Statlog Heart ===

  Neuronios=   5 | Treino=0.7259+-0.0523 | Teste=0.7284+-0.0379

  Neuronios=  10 | Treino=0.8206+-0.0309 | Teste=0.7630+-0.0339

  Neuronios=  30 | Treino=0.8720+-0.0125 | Teste=0.7728+-0.0686

  Neuronios=  50 | Treino=0.9079+-0.0177 | Teste=0.7605+-0.0359

  Neuronios= 100 | Treino=0.9608+-0.0098 | Teste=0.6988+-0.0491

  Neuronios= 300 | Treino=1.0000+-0.0000 | Teste=0.6753+-0.0371


--- Perceptron - Statlog Heart ---

  Perceptron | Treino=0.8090+-0.0415 | Teste=0.7864+-0.0503
```

Graficos - Acuracia vs Numero de Neuronios (ELM vs Perceptron)


Breast Cancer - ELM vs Perceptron
Acurácia
Número de Neurônios
Perceptron Treino (0.9776)
Perceptron Teste (0.9526)
ELM Treino
ELM Teste


Statlog Heart - ELM vs Perceptron
Acurácia
Número de Neurônios
Perceptron Treino (0.8090)
Perceptron Teste (0.7864)
ELM Treino
ELM Teste